

Costimiser AI Analytics Engine

API Documentation

Version: 1.1

Date: 23/07/2026

Status: Draft based on the current notebook usage

Base URL: <http://127.0.0.1:5000>

1. Overview

The API exposes endpoints for health checks, process-data retrieval, snapshot prediction, card-based natural-language analysis, SHAP explanations, diagnosis, cost drivers, scenarios, recommendations, optimisation, asynchronous jobs, and downloadable artifacts.

The API distinguishes between **analytical functions** and **process variables**. Analytical functions represent cost, consumption, or strength calculations. They use the following public names:

- fibre
- steam
- electricity
- starch uptake
- starch
- total
- SCT CD
- SCT MD
- Burst
- CMT30

Process variables are the actual model and process-data fields. For structured endpoints, variable names must match the canonical names exposed by **GET /process-data/variables**. Examples include `Current_basis_weight` and `Current_reel_moisture_average(reel)`, but clients should retrieve the current list rather than rely on examples copied into this document.

The exception is POST /ask-card: because its input is natural language, users may refer to variables using friendly expressions such as “current basis weight”, “starch uptake bottom”, or “starch uptake not top bottom”. The card parser resolves these expressions to canonical variables internally. This friendly-name resolution does not apply to JSON fields in the raw analytical endpoints.

2. Common request options

Asynchronous mode

Long-running POST endpoints accept:

```
{
  "async": true
}
```

When accepted asynchronously, the endpoint returns HTTP 202 and a job identifier:

```
{
  "job_id": "generated-job-id",
  "job_type": "scenario"
}
```

When `async` is false, the endpoint returns the final result directly.

Artifact mode

Analytical endpoints accept:

```
{
  "download_artifacts": true
}
```

When true, tables and figures contain artifact URLs. When false, tables and figures are returned inline.

Typical completed response:

```
{
  "text": "Markdown-formatted explanation",
  "tables": [],
  "figures": []
}
```

Downloaded tables are Parquet files. Downloaded figures are Plotly-compatible JSON.

3. Health

GET /health

Checks whether the service is available.

```
import requests

response = requests.get(
    "http://127.0.0.1:5000/health",
    timeout=30,
)
print(response.status_code)
print(response.text)
```

4. Process-data endpoints

4.1 GET /process-data/reels

Returns reels within a period.

Parameter	Type	Required
start	datetime string	Yes
end	datetime string	Yes

```
response = requests.get(
    "http://127.0.0.1:5000/process-data/reels",
    params={
        "start": "2026-03-01T00:00:00",
        "end": "2026-03-31T23:59:59",
    },
)
items = response.json()["items"]
```

4.2 GET /process-data/snapshot

Returns a process snapshot selected by either timestamp or reel.

Parameter	Type	Required
timestamp	datetime string	Conditional
reel_id	string or integer	Conditional

```
response = requests.get(
    "http://127.0.0.1:5000/process-data/snapshot",
    params={"reel_id": "12601843"},
)
snapshot = response.json()["snapshot"]
```

4.3 GET /process-data/grades

Returns the available grades.

```
grades = requests.get(
    "http://127.0.0.1:5000/process-data/grades"
).json()["grades"]
```

4.4 GET /process-data/variables

Returns the canonical process-variable names accepted by structured endpoints. The optional functions parameter filters the list to variables relevant to one or more analytical functions.

Clients should use this endpoint as the source of truth before supplying variables to /process-data/parquet, /process-data/variable-bounds, /process-data/grouped, /shap-values, /scenario, or /optimize.

```
variables = requests.get(
    "http://127.0.0.1:5000/process-data/variables",
    params={"functions": ["SCT CD", "steam"]},
).json()["variables"]
```

Examples of valid function filters include steam, SCT CD, and starch uptake.

4.5 GET /process-data/variable-bounds

Returns percentile bounds for selected internal process variables.

Parameter	Type	Required
variables	list of strings	Yes
grade	string	Conditional
reel_id	string or integer	Conditional
lower_percentile	float	Yes
upper_percentile	float	Yes

Supply either grade or reel_id.

```
bounds = requests.get(
    "http://127.0.0.1:5000/process-data/variable-bounds",
    params={
        "variables": [
            "Current_basis_weight",
            "Current_reel_moisture_average(reel)",
        ],
        "grade": "6010120",
        "lower_percentile": 0.05,
        "upper_percentile": 0.95,
    },
).json()["bounds"]
```

4.6 POST /process-data/snapshot-predictions

Evaluates friendly analytical functions using either a stored reel or supplied reference data.

Field	Type	Required
reel_id	string or integer	Conditional
reference_data	object	Conditional
functions	list of strings	Yes
cost_per_m2	Boolean	No

Supply either reel_id or reference_data.

```
{
  "reel_id": 12602792,
  "functions": ["SCT CD", "steam", "electricity"],
  "cost_per_m2": true
}
```

Response:

```
{
  "predictions": [
    {
      "function": "steam",
      "prediction": 10.5
    }
  ]
}
```

4.7 GET /process-data/parquet

Returns filtered process data directly as a Parquet file.

Confirmed query parameters include:

Parameter	Type
grade	string
start	date or datetime string
end	date or datetime string
variables	comma-separated canonical variable names

```
response = requests.get(
    "http://127.0.0.1:5000/process-data/parquet",
    params={
        "grade": "6010120",
        "start": "2026-03-01",
        "end": "2026-03-10",
        "variables": "MBS_SCT_CD,Combined_cost_€/T_",
    },
)
```

4.8 POST /process-data/grouped

Returns two pandas DataFrames: the prepared row-level process data and the grouped summary. The endpoint supports inline JSON delivery or a ZIP containing two Parquet files.

Field	Type	Required
y_variable_summary	canonical process-variable name	Yes
y_variable_summary_secondary	canonical process-variable name	No
x_variable_summary	supported summary name	Yes
color_variable_summary	supported summary name or null	No
grades	list of grade identifiers	No
target_range	two-element date list	Yes
baseline_range	two-element date list	Yes
output_format	json or parquet	No

The endpoint uses internal defaults for the aggregated cost label, the cost components to consider, and the overprocessing variables. These values are not supplied in the request.

Valid x_variable_summary values

Value	Grouping
grade	AB_Grade_ID; the grouped result also preserves grammage and paper_type
day	Wedge_Date
week	Wedge_Year and ISO Wedge_Week
month	Wedge_Year and Wedge_Month

Value	Grouping
year	Wedge_Year
target	target

Valid color_variable_summary values

Value	Grouping or transformation
null or none	No color grouping
grade	Group and color by AB_Grade_ID
target	Group and color by target
target_grade	Group by target and AB_Grade_ID
cost	Reshape the default cost components into a long-form cost column
cost_grade	Reshape cost components and group by cost and AB_Grade_ID
overprocessing	Reshape the default overprocessing variables into percentage values
overprocessing_grade	Reshape overprocessing values and group by cost and AB_Grade_ID

Weekly cost with Speed as a secondary variable

The optional `y_variable_summary_secondary` field preserves and aggregates a second process variable. In this example, the grouped result contains the weekly mean cost, weekly mean Speed, and observation count.

```
{
  "y_variable_summary": "Combined_cost_€/T_",
  "y_variable_summary_secondary": "Speed",
  "x_variable_summary": "week",
  "color_variable_summary": "cost",
  "grades": ["6010120"],
  "target_range": ["2026-05-04", "2026-05-10"],
  "baseline_range": ["2026-04-01", "2026-05-03"],
  "output_format": "parquet"
}
```

JSON output

When `output_format` is `json`, the response contains data and grouped objects using pandas split orientation. Each object contains columns, index, and data, allowing the client to reconstruct both DataFrames.

```
{
  "data": {
    "columns": ["Wedge_Time", "Speed"],
    "index": [0],
    "data": [["2026-05-04T08:00:00", 1115.0]]
  },
  "grouped": {
    "columns": ["Wedge_Year", "Wedge_Week", "cost", "Cost_€/T_", "Speed", "n"],
    "index": [0],
    "data": [[2026, 19, "Steam_€/T_", 10.8, 1115.0, 42]]
  },
  "metadata": {
    "x_variable": "Wedge_Week",
    "color_variable": "cost"
  }
}
```

Parquet output

When `output_format` is `parquet`, the HTTP response is a ZIP archive containing:

- `process_data.parquet` - prepared row-level DataFrame
- `process_grouped.parquet` - grouped summary DataFrame
- `metadata.json` - resolved x variable, color variable, color map, and row counts

The grouped endpoint returns the DataFrames directly and is therefore called synchronously by the current client helper.

5. Card-based natural-language endpoint

POST /ask-card

This is the documented natural-language entry point. /ask is intentionally excluded.

Field	Type	Required
query	string	Yes
download_artifacts	Boolean	No
diagnosis_summary	Boolean	No
cost_driver_summary	Boolean	No

```
{
  "query": "show steam cost for grade 6010120 for week 11",
  "download_artifacts": true,
  "diagnosis_summary": true,
  "cost_driver_summary": false
}
```

Variable names in /ask-card

Variable references inside query are natural-language expressions, not structured API identifiers. They may therefore be friendly and do not have to match the exact canonical field name.

- current basis weight
- starch uptake bottom
- starch uptake top
- starch uptake not top bottom

The parser uses the surrounding wording and modifiers to resolve the intended process variable. By contrast, a JSON request to /scenario or /optimize must use the exact canonical variable name returned by /process-data/variables.

Demonstrated query patterns include:

- explain model for SCT CD for grade 6010120 in April 2026
- steam cost drivers for grade 6010120 in week 18
- Diagnose cost for grade 6010120 in week 18
- simulate SCT CD for reel id 12602792 if starch uptake bottom is reduced by 10% and current basis weight is increased by 1%
- what are the recommendations for steam, grade 6010120 and week 18
- maximize SCT CD for reel id 12602391
- minimize steam cost subject to SCT CD $\geq$ 2.1 for reel id 12602391

6. SHAP values

POST /shap-values

Generates SHAP explanations for a friendly target.

Field	Type	Required
target	string	Yes

Field	Type	Required
grade	string	No
start	date string	No
end	date string	No
max_rows	integer	No
background_rows	integer	No
max_features	integer	No
variables	list of canonical names from /process-data/variables	No
async	Boolean	No
download_artifacts	Boolean	No

```
{
  "target": "SCT CD",
  "grade": "6010120",
  "start": "2026-03-01",
  "end": "2026-03-10",
  "max_rows": 100,
  "background_rows": 50,
  "async": true
}
```

A returned table may use the identifier `shap_values`.

7. Diagnosis

POST /diagnosis

Compares a target period with a baseline period.

Field	Type	Required
grade	string or null	No
target_range	two-element date list	Yes
baseline_range	two-element date list	Yes
levels	list of integers	No
objects	list of strings	No
secondary_objects	list of strings	No
summary	Boolean	No
async	Boolean	No
download_artifacts	Boolean	No

```
{
  "grade": null,
  "target_range": ["2026-04-01", "2026-04-30"],
  "baseline_range": ["2026-03-01", "2026-03-31"],
  "levels": [1, 2, 3, 4],
  "objects": ["cost"],
  "secondary_objects": ["chemicals", "steam", "electricity"],
  "summary": true,
  "async": true
}
```

8. Cost drivers

POST /cost-drivers

Explains drivers by comparing a target period with a baseline period.

Field	Type	Required
grade	string	Yes
cost_component	friendly function name	Yes
target_range	two-element date list	Yes
baseline_range	two-element date list	Yes
async	Boolean	No
download_artifacts	Boolean	No

```
{
  "grade": "6010120",
  "cost_component": "steam",
  "target_range": ["2026-04-01", "2026-04-30"],
  "baseline_range": ["2026-03-01", "2026-03-31"],
  "async": true
}
```

Demonstrated values include steam, starch uptake, and SCT CD.

9. What-if scenario

POST /scenario

Evaluates interventions against one or more friendly analytical functions.

Field	Type	Required
reel_id	string or integer	Conditional
reference_data	object	Conditional
actions	mapping of canonical variable names to proposed values	Yes
functions	list of friendly function names	Yes
cost_per_m2	Boolean	No
async	Boolean	No
download_artifacts	Boolean	No

Supply either reel_id or reference_data.

```
{
  "reel_id": "12604448",
  "actions": {
    "Current_basis_weight": 115.0
  },
  "functions": ["SCT CD", "SCT MD", "steam", "total"],
  "cost_per_m2": true,
  "async": true
}
```

Known returned table identifiers include:

- scenario_full_snapshot
- scenario_function_evaluation

10. Recommendations

POST /recommendations

Generates recommendations for a grade and cost component.

Field	Type	Required
grade	string	Yes

Field	Type	Required
cost_component	friendly function name	Yes
target_range	two-element date list	Yes
baseline_range	two-element date list	Yes
async	Boolean	No
download_artifacts	Boolean	No

```
{
  "grade": "6010120",
  "cost_component": "starch uptake",
  "target_range": ["2026-04-01", "2026-05-03"],
  "baseline_range": ["2026-05-04", "2026-05-10"],
  "async": true
}
```

Demonstrated values include steam, starch, and starch uptake.

11. Optimisation

POST /optimize

Optimises a friendly analytical function from either a reel or supplied reference data.

Field	Type	Required
reel_id	string or integer	Conditional
reference_data	object	Conditional
objective_function	friendly function name	Yes
direction	minimize or maximize	Yes
constraints	object, list, or null	No
candidate_features	canonical variable-name list	No
exclude_features	canonical variable-name list	No
max_interventions	integer	No
overprocessing	Boolean	No
cost_per_m2	Boolean	No
async	Boolean	No
download_artifacts	Boolean	No

Compact constraint form:

```
{
  "SCT CD": 2.05,
  "SCT MD": 4
}
```

Explicit constraint form:

```
[
  {
    "function": "SCT CD",
    "operator": ">=",
    "value": 1.95
  },
  {
    "function": "Burst",
    "operator": ">=",
    "value": 240
  }
]
```

Full example:

```
{
  "reel_id": "12604077",
  "objective_function": "total",
  "direction": "minimize",
  "constraints": [
    {
      "function": "SCT CD",
      "operator": ">=",
      "value": 1.95
    },
    {
      "function": "Burst",
      "operator": ">=",
      "value": 240
    }
  ],
  "candidate_features": [
    "Current_basis_weight",
    "Starch_uptake_by_paper_Bottom_Roll__g/m2_"
  ],
  "max_interventions": 2,
  "overprocessing": true,
  "cost_per_m2": true,
  "async": true
}
```

Demonstrated objective functions include steam, total, starch, starch uptake, and SCT CD.

Returned optimisation table IDs may be generic names such as block_4, block_5, or block_6. Clients should use the IDs returned by the API rather than assume fixed names.

12. Job status

GET /jobs/{job_id}

Recognised states are:

- queued
- running
- completed
- failed

Queued/running response:

```
{
  "job_id": "generated-job-id",
  "status": "running"
}
```

Completed response:

```
{
  "job_id": "generated-job-id",
  "status": "completed",
  "result": {
    "text": "Analysis completed",
    "tables": [],
    "figures": []
  }
}
```

Failed response:

```
{
  "job_id": "generated-job-id",
  "status": "failed",
  "error": "Description of the error"
}
```

A failed response may also contain a partial or diagnostic result.

13. Current job-management limitations

The current asynchronous interface provides submission and polling, but not a complete job-management system.

- Clients must retain the returned job_id.
- Completion is detected by polling GET /jobs/{job_id}.
- No callback or webhook endpoint is documented.
- No job-cancellation endpoint is documented.
- No endpoint for listing jobs is documented.
- No endpoint for deleting jobs is documented.
- No percentage-complete or detailed progress field is documented.
- Progress is limited to queued, running, completed, and failed.
- No job-priority mechanism is documented.
- No idempotency key is documented; repeated submissions may create independent jobs.
- Job-retention and artifact-retention periods are not part of the documented contract.
- Authentication, ownership, and per-user job access are not represented in the demonstrated schemas.

The notebook does not establish whether job state survives a service restart or whether it is shared across several pods. These points should remain documented as unknown until the Flask implementation is checked.

14. Artifact retrieval

When download_artifacts is true, use the returned artifact URL exactly as supplied.

Table:

```
import io
import pandas as pd
import requests

response = requests.get(
    base_url + table["artifact"]["url"],
    timeout=120,
)
response.raise_for_status()
df = pd.read_parquet(io.BytesIO(response.content))
```

Figure:

```
import plotly.graph_objects as go
import requests

response = requests.get(
    base_url + figure["artifact"]["url"],
    timeout=120,
)
response.raise_for_status()
fig = go.Figure(response.json())
```

15. Naming rules summary

Context	Name type required	Example
target, functions, cost_component, objective_function, constraint function	Analytical function name	steam, starch uptake, SCT CD
Raw endpoint variable fields such as variables, actions, candidate_features, and exclude_features	Exact canonical process-variable name from /process-data/variables	Current_basis_weight

Context	Name type required	Example
Natural-language text in /ask-card	Friendly variable wording accepted and resolved by the card parser	current basis weight, starch uptake bottom

Do not use friendly variable wording in structured JSON fields unless the endpoint explicitly documents that behavior.

16. Endpoint summary

Method	Endpoint	Purpose
GET	/health	Service health
GET	/process-data/reels	List reels
GET	/process-data/snapshot	Retrieve a process snapshot
GET	/process-data/grades	List grades
GET	/process-data/variables	List variables
GET	/process-data/variable-bounds	Retrieve percentile bounds
POST	/process-data/snapshot-predictions	Predict functions for a snapshot
GET	/process-data/parquet	Download process data
POST	/process-data/grouped	Prepared and grouped process DataFrames
POST	/ask-card	Card-based natural-language analysis
POST	/shap-values	SHAP explanations
POST	/diagnosis	Diagnosis
POST	/cost-drivers	Cost-driver analysis
POST	/scenario	What-if scenario
POST	/recommendations	Recommendations
POST	/optimize	Optimisation
GET	/jobs/{job_id}	Job status and result
GET	Returned artifact URL	Download a table or figure

17. Items not established by the notebook

The notebook does not establish:

- Authentication or authorisation headers
- Rate limits
- Maximum request sizes
- Formal OpenAPI schemas
- Exact validation rules for every optional parameter
- Job and artifact retention periods
- Restart recovery
- Multi-pod job-state behaviour
- Complete error-code mappings

These items should not be described as implemented behavior without checking the Flask source.